

Stellar Mall

AI 系统面试题集

覆盖 RAG + Agent 两大模块

四大层级：基础 / 进阶 / 深水区 / 开放题

共 19 道面试题 + 详细解析

Stellar Mall Project - rag-backend

章节 1 基础层 — 概念理解

[QQ1] (基础)

请描述一下 RAG（检索增强生成）的基本工作流程，在这个项目中 RAG 的核心链路是怎样的？

AA1

RAG 的基本流程：用户提问 → 检索相关文档 → 将检索结果作为上下文注入 Prompt → LLM 生成答案。

本项目中的 RAG 核心链路：

用户问题 → [查询改写(若开启)] → [混合检索(向量+BM25)] → [精排(Rerank)] → [Prompt组装] → [LLM流式生成] → [缓存写入]

具体实现在 app/rag/chains.py 的 astream_answer_with_sources 函数中，以 SSE（Server-Sent Events）流式输出结果。

[QQ2] (基础)

项目中使用了哪几种检索方式？它们各自的权重和角色是什么？

AA2

项目使用了混合检索（Hybrid Retrieval），包含两种方式：

1. 向量检索（ChromaDB，默认 top_k=20）：基于语义相似度
2. BM25 关键词检索（rank-bm25 库）：基于关键词词频统计

两者通过 EnsembleRetriever 融合，权重比例为：

向量检索：0.6（主，语义匹配）

BM25：0.4（辅，关键词匹配）

向量检索权重更高，因为对于电商客服场景，语义理解比关键词精确匹配更重要。

[QQ3] (基础)

文本切分（Chunking）在这个项目中是如何配置的？为什么需要设置 chunk overlap？

AA3

切分配置：

chunk_size = 512（每个片段约 512 个字符）

chunk_overlap = 64（相邻片段重叠 64 个字符）

分隔符优先级：["\n\n", "\n", "。", "!", " ", "? ", ":", "; ", " ", " ", ""]

设置 overlap 的原因：如果一个语义完整的句子被切分边界截断，overlap 可以确保关键信息在两个相邻 chunk 中都出现，避免检索时遗漏重要上下文。

[QQ4] (基础)

项目中使用了哪几种 Embedding 模型？它们之间是什么关系（主备策略）？

AA4

项目使用了三层的 Embedding 模型兜底策略：

1. DashScope API Key (sk-xxx) → DashScopeEmbeddings (模型: text-embedding-v2)
2. 业务空间 Key (sk-ws-xxx) → OpenAIEmbeddings (兼容模式走 /v1/embeddings)
3. 兜底 (云端均失败) → HuggingFaceBgeEmbeddings (BAAI/bge-large-zh-v1.5, 1024维)

采用单例模式，优先级为：云端 > 本地，因为云端的 QA 模型向量质量优于本地开源模型。

[QQ5] (基础)

Agent 系统支持哪些意图分类？请列举至少 8 种。

AA5

项目支持 16 种意图：

1. product_consult — 商品咨询 (走知识库 RAG)
2. product_search — 商品搜索 (走 Mall API)
3. order_query — 订单查询
4. cancel_order — 取消订单
5. confirm_receipt — 确认收货
6. cart_query — 购物车查看
7. cart_add — 加入购物车
8. cart_clear — 清空购物车
9. cart_delete — 删除购物车商品
10. cart_update — 修改购物车商品
11. after_sales — 售后服务
12. favorite_query — 收藏夹查询
13. wallet_query — 钱包查询
14. review_query — 商品评价
15. small_talk — 闲聊
16. other — 其他

章节 2 进阶层 — 设计与实现

[QQ6] (进阶)

项目中 Agent 系统使用了 LangGraph 状态图。请描述 Agent 的状态流转图，说明各个节点的作用。

AA6

Agent 状态流转：

```
入口 → intent_classification (意图识别) → 路由判断
|
| — check_params (检查参数是否齐全)
| |
| | — ask_params (向用户追问缺失参数) → END
| | — execute_tool (调用具体工具) → generate_answer (组装回答) → END
|
| — small_talk (闲聊) → generate_answer → END
|
| — other (需要工具的其他意图) → execute_tool → generate_answer → END
```

各节点作用：

intent_classification：调用 LLM 识别用户意图，输出结构化 JSON

check_params：检查工具调用所需参数是否齐全

ask_params：向用户发起追问（如“请问您要取消哪个订单？”）

execute_tool：执行工具函数（查订单、加购等）

generate_answer：将工具执行结果组装成自然语言回答

[QQ7] (进阶)

RAG 的查询改写（Query Rewrite）为什么是必要的？它是如何工作的？

AA7

为什么需要查询改写？

在多轮对话中，用户经常使用指代（如“它的价格是多少？”“帮我退了这个”），这些指代如果不处理，直接检索会导致语义偏移。

工作原理：

1. 仅在开启 QUERY_REWRITE_ENABLED=true 且存在多轮历史上下文时触发
2. 调用 LLM，传入最近的对话历史和当前问题
3. LLM 将带指代/省略的问题改写为完整的独立查询语句
4. 改写后的查询用于向量检索

示例：

用户：“华为P60多少钱？” 助理：“华为P60售价为4499元起。” 用户：“那Mate60呢？” → 改写为
“华为Mate60多少钱？”

[QQ8] (进阶)

混合检索中的 EnsembleRetriever 是如何融合向量检索和 BM25 结果的？项目中设置的权重比例是多少？

AA8

EnsembleRetriever 的融合策略：

向量检索权重：0.6（主，语义匹配）

BM25 检索权重：0.4（辅，关键词匹配）

融合方式：使用 LangChain 的 EnsembleRetriever 类，它会：

1. 分别从两个检索器获取候选文档
2. 每个文档的最终得分 = $0.6 \times \text{向量得分} + 0.4 \times \text{BM25 得分}$
3. 按得分降序取 top_k

好处：向量检索能理解语义相似但用词不同的查询，BM25 能精确匹配关键商品名/型号。

[QQ9] (进阶)

精排（Rerank）步骤具体做了什么？相似度阈值（SIMILARITY_THRESHOLD）的作用是什么？如果所有结果都低于阈值会怎样？

AA9

精排步骤：

1. 获取混合检索返回的候选文档（默认 top_k=20）
2. 用 Embedding API 对查询和每个文档计算向量
3. 计算余弦相似度，按降序排列
4. 截取 top_k=5（RERANK_TOP_K）

相似度阈值（SIMILARITY_THRESHOLD=0.4）的作用：

相似度 < 0.4 的文档被过滤，避免低质量上下文输入 LLM。

特殊处理：如果所有文档都低于阈值，系统会至少保留最相关的一篇，确保 LLM 回答时不是零上下文。这是一种容错设计。

[QQ10] (进阶)

Agent 的工具调用是如何与项目中的 Java 后端（Mall Backend）交互的？熔断器（Circuit Breaker）的配置参数是怎样的？

AA10

Agent 工具与 Mall Java 后端通过 HTTP REST API 交互。

统一客户端（app/agent/tools/mall_client.py）封装了请求逻辑：

请求头携带三层认证：Authentication、Authorization: Bearer、stellar-token

基础 URL：MALL_API_BASE_URL=http://127.0.0.1:8082

熔断器参数：

失败阈值：连续 5 次失败 → 熔断器 OPEN

开路持续时间：30 秒

半开恢复：30 秒后进入 HALF_OPEN，允许一个试探请求

重试策略:

最多重试 2 次

退避间隔: $0.2 \times \text{retry_num}$ 秒

仅对 5xx / Timeout / ConnectError / NetworkError 重试

章节 3 深水区 — 架构与安全

[QQ11] (深水区)

RAG 后端如何实现跨端单点登录 (SSO) ? “三段式 JWT 解码” 具体指什么?

AA11

RAG 后端实现了与 Mall Java 后端的跨端 SSO。

三段式 JWT 解码 (app/core/security.py) :

按顺序尝试三个不同的密钥解析 JWT:

1. RAG 自有 SECRET_KEY → 标记 _src=rag
2. Mall 管理端 STELLAR_ADMIN_SECRET_KEY → 标记 _src=stellar_admin
3. Mall 用户端 STELLAR_USER_SECRET_KEY → 标记 _src=stellar_user

映射机制 (影子用户) :

Mall 签发的 JWT 被 RAG 解码后, 在 users 表中查找或创建同名用户

管理端 token → ADMIN 角色, C 端 token → USER 角色

这样用户在 Mall 登录后, 其 JWT 可直接用于 RAG 服务的鉴权, 无需二次登录。

[QQ12] (深水区)

项目中 /api/internal/sync_spu 和 /api/internal/sync_doc 这两个内部同步接口的作用是什么? 它们解决了什么问题?

AA12

解决的问题: Mall Java 后端管理后台修改商品信息、发布商品后, RAG 系统的知识库能自动感知并同步更新。

/api/internal/sync_spu:

调用方: Mall 后端 (商品发布/修改时触发)

鉴权: 共享密钥 X-Stellar-Rag-Sync-Secret

功能: 将 SPU 商品信息转化为文档, 写入 ChromaDB

/api/internal/sync_doc:

调用方: Mall 后端

功能: 同步运营人员上传的文档 (活动规则、售后政策等) 到知识库

这两个接口是 Mall RAG 的数据桥接, 使得商品信息变更后无需手动重新导入知识库。

[QQ13] (深水区)

纯 RAG 和 Agent 两种模式在设计上各有什么优缺点? 项目中是如何在路由层做模式切换的?

AA13

RAG 模式:

优点: 简单、快速、适合纯知识问答

缺点: 只能回答知识库中已有内容

适用场景: 商品咨询、政策说明

幻觉风险：有检索上下文，幻觉较低

Agent 模式：

优点：功能强大，可执行操作（查订单/加购/退款）

缺点：延迟更高（需多步推理），复杂度高

适用场景：售后操作、订单管理、多步任务

模式切换（/api/chat 路由）：

请求参数 use_agent: true → Agent 模式, false → RAG 模式, null → 系统默认配置 (AGENT_ENABLED)

区别：RAG 模式走 rag_chat_service → chains.astream_answer_with_sources

Agent 模式走 agent_service → graph.astream

[QQ14] (深水区)

查询缓存（Query Cache）的命中条件是什么？如果知识库发生了变更，缓存如何处理？

AA14

命中条件（_QueryCache）：

将当前问题用 Embedding 转为向量

与缓存中已有问题的向量计算余弦相似度

相似度阈值：0.97（非常高，几乎完全相同的问题才命中）

缓存大小限制：maxsize=128

TTL：300 秒（5分钟内有效）

知识库变更处理：

当知识库发生任何变更时，调用 cache.clear()

清空所有缓存条目，防止返回过时的回答

设计理念：适合电商常见的“同款比价”场景，同一问题 5 分钟内重复问可直接命中缓存节省一次 LLM 调用。

[QQ15] (深水区)

限流（Rate Limiting）在这个项目中是如何实现的？不同接口的限流阈值分别是多少？

AA15

项目使用内存滑动窗口限流算法（app/core/rate_limiter.py），基于滑动窗口记录每个客户端的请求时间戳。

限流阈值：

/api/auth/register: 60 次/分钟（按 IP）

/api/auth/login: 60 次/分钟（按 IP）

/api/auth/change-password: 20 次/分钟（按 IP）

/api/chat: 15 次/分钟（按用户）

实现细节：

使用 defaultdict(list) 存储每个 key 的时间戳列表

请求到达时，移除窗口外的过期时间戳

检查剩余时间戳数量是否超过阈值

章节 4 开放题 — 架构权衡与改进

[QQ16] (开放题)

项目使用 ChromaDB 作为向量数据库。如果你要重新选型，你会考虑哪些替代方案（如 Milvus、Pinecone、PGVector）？各自的适用场景是什么？

AA16

向量数据库对比：

ChromaDB（当前）：适用单机/小规模

优点：轻量、零运维

缺点：不支持分布式

Milvus：适用百万级以上向量、生产环境

优点：分布式、GPU加速、多索引

缺点：需部署集群

PGVector (PostgreSQL)：适用已有 PG、中小规模

优点：无需额外组件、支持 SQL 联合查询

缺点：性能不如专用向量库

Pinecone：适用不想自建基础设施

优点：全托管、零运维

缺点：云成本高、数据需外传

建议：数据量在百万以内升级到 PGVector 或 Milvus。

[QQ17] (开放题)

当前 Agent 系统在处理复杂多步任务时，如果某个工具调用失败（如查订单超时），Agent 的行为是怎样的？你会如何优化它的容错策略？

AA17

当前行为：

熔断器 + 重试机制处理暂时性故障（最多重试 2 次，退避等候）

最终失败时，工具返回错误消息，Agent 将错误转化为自然语言告知用户

优化建议：

1. 降级策略：API 连不上时降级到本地缓存查询
2. 自我纠正：工具返回空结果时，Agent 重新解析意图换个方式重试
3. 用户确认：敏感操作前展示摘要并请求确认
4. 超时分层：工具级别超时 < Agent 级别超时 < 请求级别超时
5. 局部状态回滚：多步任务中间某步失败时回滚已执行操作

[QQ18] (开放题)

RAG

使用

RecursiveCharacterTextSplitter

按固定大小（512）切分，这可能导致语义不完整的片段。你有什么优化建议？

AA18

问题分析：

固定 512 字符切分可能导致：

商品完整描述被切成两半

相关字段被分散到不同 chunk

检索时只命中一半信息

优化建议：

1. 语义切分（Semantic Chunking）：按语义完整性切分
2. 分字段结构化存储：商品标题、描述、规格参数分开存储，检索时按字段加权
3. 滑动窗口重叠增强：使用更大的 overlap
4. Parent-Child 检索（推荐）：小块用于检索，大块用作 LLM 上下文
5. LLM 辅助切分：用 LLM 判断自然断点

[QQ19] (开放题)

如果用户量级增长到日活 10 万，你认为当前架构的瓶颈可能出现在哪些环节？你会如何设计扩容方案？

AA19

可能的瓶颈：

1. LLM API 调用：单点依赖通义千问
2. ChromaDB：单机持久化，不支持并发写
3. 内存缓存：问题分布变广，128 的 LRU 兜不住
4. 熔断器：Mall API 压力增大导致频繁 OPEN
5. SQLite：高并发写锁
6. 单进程 FastAPI：Python GIL

扩容方案：

1. LLM 层：多模型负载均衡，本地部署小模型降级
2. 向量库：升级 Milvus 支持分布式分片
3. 缓存：引入 Redis，L1（本地LRU）+ L2（Redis集群）
4. 数据层：SQLite → PostgreSQL + 读写分离
5. 应用层：FastAPI 多 worker + 水平扩展多实例
6. 限流：单机内存 → Redis 分布式限流
7. 异步化：耗时操作放入 Celery/RabbitMQ 任务队列